

Profiles & connections

Nada helps families discover marriage connections through the people a family already knows. Build a two-screen Flutter app that lets someone browse profiles and see how each person is connected.

Due: Saturday, 10 October 2026, 9 pm IST

Expected effort: an evening or two.

01 Load the supplied profiles

Use this shared dataset of **24 profiles**: [Open the raw profiles JSON](#). The same clickable URL is included in the assignment email.

Load the file from that HTTPS link with a network call, rather than bundling a local copy into the app. The link serves plain text; decode the response body as JSON yourself. The top-level object contains a **profiles** array.

02 Screen one: profile list

Show one row per profile with **name**, **age**, **city**, and the **connected_through** line.

Place a search field at the top. Filter by **name and city** as the user types. Matching must be case-insensitive.

Show a **loading state** while fetching the file, an **empty state** that says "No profiles match", and an **error state with a retry button** when the request fails.

03 Screen two: profile details

Open a profile on tap. Show every field that is present, make the **connected_through** line prominent, and provide a way to go back.

When a profile has no connection yet, say so plainly. Handle missing or null values without crashing or showing confusing placeholder text.

04 Handle the supplied data

A few profiles contain missing or null fields, one has no connection yet, some text is long, and one profile contains Hindi. Every profile must render without crashes or text overflow.

Test, document & send back

State management and test

Riverpod is preferred; any state-management approach is accepted. Keep widget responsibilities clear and state handling separate from rendering. Avoid putting application logic inside build.

Include **at least one widget test**, for example that typing in the search field filters the list, or that an unmatched query shows the empty state.

Check before submitting

Run the app on an Android emulator or an iOS simulator. Check the list, search, profile navigation, loading state, empty state, and error/retry behaviour.

To exercise a failed request, start the app offline so the initial fetch fails, or temporarily use an invalid URL. Restore connectivity or the correct URL and retry. Submit with the supplied dataset URL restored.

How to submit

Reply to the assignment email with your **GitHub repository link** by **Saturday, 10 October 2026, at 9 pm IST**.

Your repository may be public or private. If private, invite **both harshtitoria-dev and jordandivyansh** as collaborators before submitting.

Include a README covering:

1. How to run the app on an Android emulator or iOS simulator.
2. How to run the widget test.
3. What you would do next with more time.
4. Which AI tools you used, if any.

Commit as you go. We read the commit history. AI tools are welcome; use what you would use at work and disclose them in the README.

What we review

We run the app from the README and review loading, empty, and error states; data handling; widget structure; where state lives; the widget test; and commit history.

The interview with the founder starts from your submission. You will be asked to make one change live on a screen-share.

Questions? Reply to the assignment email.